# Data Engineering & Introduction to Databases

**Lab Guide: Modern Project Configuration with `pyproject.toml`**

WS 2026 / 2027

## 1. Motivation & Background

In professional data pipelines, hard-coded dependencies and fragmented setup scripts lead to non-deterministic builds and deployment issues. Historically, Python projects required multiple disjoint configuration files (`setup.py`, `setup.cfg`, `requirements.txt`).

The standardized **`pyproject.toml`** specification (**PEP 518** and **PEP 621**) replaces this legacy model with a single, declarative, and secure configuration format. It prevents arbitrary code execution at build time and defines exact runtime dependencies for every team member.

## 2. Structural Overview

A clean and compliant `pyproject.toml` divides configuration into distinct tables:

| Table Section | Standard | Purpose & Responsibilities |
|---|---|---|
| **`[build-system]`** | PEP 518 | Declares the packaging engine (e.g., `setuptools`) needed to assemble and install the project. |
| **`[project]`** | PEP 621 | Formal metadata: project title, release version, maintainer, and supported Python version ($\geq$ 3.14). |
| **`dependencies`** | PEP 621 | Declares the core libraries required to execute the pipeline (e.g., `requests`, `pandas`). |
| **`optional-dependencies`** | PEP 621 | Isolated extras for local quality control and unit testing (e.g., `pytest`, `ruff`). |

## 3. Recommended VS Code Extension: Even Better TOML

To prevent syntax errors and enable autocompletion, configure your editor with the following extension:

- **Extension Name: Even Better TOML** (`tamasfe.even-better-toml`)
- **Schema Validation:** Automatically pulls the official PEP 621 JSON schema from SchemaStore to highlight unknown keys or incorrect types.
- **IntelliSense & Tooltips:** Hovering over tables like `[build-system]` or `[project]` displays the exact specification rules and valid parameter types directly in VS Code.

## 4. Project Starter Template

Save the following minimal configuration directly in the root directory of your lab repository:

```
# VS Code Tip: Install "Even Better TOML" (tamasfe.even-better-toml)
# for automated PEP 621 schema validation and auto-completion.

[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "wspr-data-pipeline"
version = "0.1.0"
description = "WSPR propagation telemetry ingestion and database pipeline"
authors = [{ name = "Prof. Dr. Peter Kern" }]
requires-python = ">=3.14"
dependencies = [
    "requests>=2.31.0",
    "pandas>=2.2.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
    "ruff>=0.4.0",
]
```

## 5. Practical Lab Execution

Once your virtual environment (`.venv`) is active, install the pipeline in **editable mode** (`-e`) so local script modifications take effect immediately without reinstalling:

```
# 1. Activate your virtual environment (.venv)
# 2. Install project dependencies including developer tooling:
pip install -e ".[dev]"

# 3. Verify the setup by running the code quality checker:
ruff check .
```

**Key Distinction: `pip install .` vs. `pip install -e .`**

**Standard Install (`pip install .`):** Copies your code snapshot directly into `.venv/lib/.../site-packages`. Any subsequent changes you make to your local Python source files are **not** reflected at runtime until you manually re-run the installation command.

**Editable Install (`pip install -e .`):** Creates a direct symbolic link (`.pth` file) in `site-packages` that points to your local project directory. Code modifications take effect **immediately** without re-running pip—essential for iterative script development and pipeline debugging.